Что реализовано сверх технического задания

Документ отвечает на вопрос жюри «что вы сделали сверх задания». Обязательные требования R1-R6 закрыты и перечислены в ROADMAP.md и SUBMISSION.md; здесь — только то, что выходит за их рамки.

Разделение намеренное. Совпадение правила с примерами не является доказательством, и точно так же обязательный пункт не становится заслугой от того, что назван таковой. Поэтому в конце есть раздел «Что честно НЕ сверх ТЗ».

Область этого документа ограничена автором: модули `src/protocol/` и `src/hypothesis/`. Для остальных модулей оставлены плейсхолдеры, которые заполняют их авторы.

1. Модуль `src/hypothesis/` — анализ гипотез

Возможность	Файл	Зачем
Движок альтернативных прочтений: для поля-гипотезы строятся конкурирующие чтения (`low_cardinality`, `counter`, `constant`, `message_length`, `entropy_enum`, `entropy_parameter`, `entropy_random`) и оцениваются на корпусе	`src/hypothesis/alternatives.py`	Показывает не «правило совпало», а какое из нескольких прочтений лучше подтверждается корпусом, и где — нет
Метрики качества правила: `precision`, `coverage`, `counterexample_density`	`src/hypothesis/metrics.py`	Даёт числовую оценку интерпретации вместо бинарного «работает / не работает»
Связывание запроса и ответа в ходы с `confidence` и `basis` (перечень наблюдений, на которых держится уверенность)	`src/hypothesis/pairing.py`	Отвечает на вопрос «что отвечает на что»; в `ROADMAP.md` отнесено к «Out of MVP scope» и всё равно реализовано
Корреляция сообщений с журналом действий по временному окну	`src/hypothesis/journal.py`	Позволяет связать байты с событием в приложении (500 мс окно по умолчанию)

2. Модуль `src/protocol/` — правила и фрейминг

Возможность	Файл	Зачем
19 типов полей вместо шести в ТЗ: `float32/64`, `varint` (со знаковым zig-zag), `tlv`, `bitmask`, `string`, `array`, `padding`, `checksum` (xor/sum/crc8/crc16), `computed`, `conditional`	`src/protocol/rule.py`, `docs/schemas/rule.schema.json`	Недокументированный протокол редко состоит из одних целых; типы снимают ручное описание
`length_covers` (`payload` / `payload_and_length_field` / `entire_message`) и `length_bias`	`src/protocol/rule.py`	Снимает разницу между тем, что именно считает поле длины, без правки кода
Экспорт правила в Kaitai Struct `.ksy` и в автономный Python-парсер	`src/protocol/export.py`; команда `export`	Полученное знание уезжает из проекта как независимый артефакт и переиспользуется вне инструмента
Выгрузка результата в CSV и JSONL	`src/protocol/cli.py`	Машинная обработка результата вне приложения
Экспорт собранных потоков обратно в pcap для сверки с Wireshark `Follow TCP Stream`	`src/protocol/`; `--wireshark-pcap` в захвате	Позволяет проверить сборку сторонним инструментом, а не только своим кодом

3. Вне зоны этого документа

Документ закрывает сквозной сценарий и его обоснование. Разбор внутреннего устройства отдельных модулей, границ ответственности и деталей реализации — в `ARCHITECTURE.md` и `docs/PROJECT_MAP.md`.

4. Что честно НЕ сверх ТЗ

Эти пункты обязательны и закрыты, но они — не заслуга, а требования:

Пункт	Раздел ТЗ
Четыре стратегии фрейминга: length-prefixed, fixed-size, marker-based, manual	R3
JSON-контракты между стадиями (`capture.schema.json`, `result.schema.json`, `rule.schema.json`)	R5, R6
Шесть основных статусов: `matched`, `mismatched`, `incomplete`, `ambiguous`, `uncovered`, `not_applicable`	R4
Версионирование правил и пометка старых результатов как `outdated`	R3, R4
Проверка правила на корпусе с поиском контрпримеров	R4

Статусов в реализации восемь: к шести обязательным добавлены `outdated` и `unknown`, но это расширение словаря, а не отдельная функциональность сверх задания.

5. Ссылки

- PROJECT_MAP.md — карта проекта.
- ROADMAP.md — требования R1-R6.

- SUBMISSION.md — что закрыто из ТЗ.